

动态规划与回溯算法求解性能比较分析

算法分析与设计 实验报告 (2.2 算法验证类)

学	院	： 智能与计算学部
专	业	： 计算机科学与技术
年	级	： 2023 级
姓	名	： 朱发楷
学	号	： 3023207254
分	组	： 朱发楷

天津大学 智能与计算学部

2026 年 6 月

摘要

本实验针对 0/1 背包问题、子集和数问题、矩阵乘法链问题，分别采用动态规划与回溯算法，各选取 4 组差异显著的数据集进行实验验证。数据来源于 GitHub 公开数据集(TJU-NSL/Datasets-Course)，包含 Florida State University 和 Unicauca University 的标准测试实例。通过对比不同数据集特征下两种算法的运行时间和搜索效率，系统分析了数据分布特点对算法性能的影响。实验结果表明：动态规划对容量/目标和参数敏感($O(nC)$ 或 $O(nt)$)，但对数据分布不敏感，性能稳定；回溯法通过剪枝可在大容量数据上远超动态规划(如 p08 快 12,654 倍)，但在强相关实例中剪枝近乎失效；矩阵乘法链中回溯法因 Catalan 数级搜索空间在 $n \geq 32$ 时完全不可行，DP 始终高效。

一、实验目的

- 1. 掌握动态规划算法（自底向上填表策略）与回溯算法（深度优先搜索结合剪枝技术）的编码实现方法。
- 2. 通过标准化测试数据集验证两种算法在各类问题中的正确性，并将其结果与标准答案进行对比分析。
- 3. 探究数据集的分布特性（包括问题规模 n 、容量或目标值 C/t 、以及数据间的相关性）对算法运行效率与求解能力的影响。
- 4. 通过对比实验，系统归纳动态规划与回溯算法在不同问题背景下的适用范围、性能表现及各自优势与局限。

二、实验设计流程

2.1 算法设计思路

动态规划算法的基本原理在于利用最优子结构特性，将原问题分解为若干具有重叠性质的子问题，并通过自底向上的方式逐层构建最优解。在实现过程中，采用滚动数组策略以优化空间复杂度。

回溯算法的核心思想是基于深度优先搜索策略对解空间树进行系统性遍历，通过引入约束函数与限界函数，有效剪除无法生成最优解的分支，从而提高搜索效率。在具体实现中，结合启发式排序方法与多种剪枝策略，以增强算法的实用性与性能表现。

2.2 数据集选择

2.2.1 0/1 背包问题 (从 38 个数据集中筛选)

数据集	n	容量	数据特征	选择理由
p02 (FSU)	5	26	小规模，容量适中	小规模基准测试，验证算法基本正确性
p08 (FSU)	24	6,404,180	超大容量，远超物品总重	考察容量 C 极大时 DP 的 $O(nC)$ 退化程度，预期回溯剪枝优势
knapPI_3_1000	1000	4,990	强相关实例(Pisinger type3)	考察强相关数据对回溯剪枝的破坏性影响，预期 DP 反超
knapPI_1_5000	5000	25,016	大规模非相关 (type1)	与 DS3 形成 n 相同/相关性相反的对偶实验，验证数据分布影响

2.2.2 子集和数问题 (从 7 个数据集中筛选)

数据集	n	目标和 t	总和	数据特征	选择理由
p06	6	22	63	最小规模, t 极小	最小规模基线, 验证算法基本正确性
p04	10	50	148	n=10, t 远小于总和	考察小 t 场景, 预期两者均快速完成
p02	10	5,842	11,275	n=10, t 与总和相当	与 p04 同 n 不同 t 的对照实验, 考察 t 增大对 DP 的 O(nt)影响
p03	21	2,463,098	19,499,483	极端大 t(2.46×10^6)	考察极端大 t 场景下 DP 的性能瓶颈, 而回溯预期对 t 不敏感

2.2.3 矩阵乘法链问题 (从 9 个数据集中筛选)

数据集	n	维度范围	搜索空间 Catalan (n-1)	数据特征	选择理由
input_004	4	[2,14]	5	最小链长, 可手工验证	最小规模基准, 搜索空间仅 5 种括号化
input_008	8	[1,16]	429	小链长, 回溯可穷举	回溯仍可完成, 展示剪枝对 429 种方案的削减效果
input_016	16	[2,13]	9.69×10^6	中等链长, 搜索空间近千万	预期回溯需大量剪枝才能完成, DP 的 O(n^3)稳定优势开始显现
input_032	32	[1,16]	1.45×10^16	大链长, 搜索空间天文数字	回溯理论上不可行(Catalan 数爆炸), 预期 DP 毫秒级求解, 形成终极对比

2.3 实验评价指标

- 运行时间 (ms)：采用高精度 `chrono` 计时器，记录算法核心计算阶段的耗时情况；
- 基本操作数：动态规划算法中内层循环的执行次数，或回溯算法中搜索节点的总数；
- 最优解值：算法求解所得的最优目标函数值；
- 标准答案比对：通过与数据集中预置的最优解文件进行交叉验证，评估算法求解精度；
- 算法一致性：对动态规划与回溯算法的求解结果进行相互验证，以确保结果的可靠性与一致性。

三、代码实现

以下给出六个算法的伪代码与关键 C++ 代码。完整工程源代码见附件。

3.1 0/1 背包问题 — 动态规划

算法思想：

定义 $dp[c]$ 为容量 c 时的最大价值，状态转移方程 $dp[c] = \max(dp[c], dp[c - w_i] + v_i)$ 。一维滚动数组将空间从 $O(nC)$ 降至 $O(C)$ ，逆序遍历保证 0/1 性质。 $O(nC)$ 时间。

伪代码：

算法 Knapsack_DP(weights[1..n], values[1..n], capacity)

1. $dp[0..capacity] = 0$
2. for $i = 1$ to n do
3. for $c = capacity$ downto $weights[i]$ do
4. $dp[c] = \max(dp[c], dp[c - weights[i]] + values[i])$
5. return $dp[capacity]$

关键代码(C++)：

```
// 一维滚动数组 DP， $O(n \cdot C)$  时间， $O(C)$  空间
vector<long long> dp(capacity + 1, 0);
for (int i = 0; i < n; i++) {
    long long w = weights[i], v = values[i];
    for (long long c = capacity; c >= w; c--) // 逆序保证 0/1
        dp[c] = max(dp[c], dp[c - w] + v);
}
return dp[capacity];
```

3.2 0/1 背包问题 — 回溯法

算法思想：

物品按价值密度 (v_i/w_i) 降序排列，DFS 搜索解空间树。上界函数采用分数背包贪心解： $bound = \text{当前价值} + \text{剩余容量} \times \text{剩余最大单位价值}$ 。若 $bound \leq best$ 则剪枝。

伪代码：

算法 Knapsack_Backtrack(items[1..n], C)

1. items = 按 v_i/w_i 降序排列
2. $best = 0$
3. procedure DFS(idx, cur_w, cur_v)
4. if $cur_v > best$ then $best = cur_v$
5. if $idx > n$ then return
6. $bound = cur_v + \text{FractionalUB}(idx, C - cur_w)$ // 分数背包上界
7. if $bound \leq best$ then return // 上界剪枝
8. if $cur_w + w[idx] \leq C$ then

9. DFS(idx+1, cur_w+w[idx], cur_v+v[idx]) //左分支:选
10. DFS(idx+1, cur_w, cur_v) //右分支:不选
11. DFS(1, 0, 0)
12. return best

关键代码(C++):

```
// 分数背包上界剪枝(核心)
long long rem = capacity - cur_w, bound = cur_v, tmp = 0;
for (int i = idx; i < n && tmp < rem; i++) {
    if (items[i].weight <= rem - tmp) {
        bound += items[i].value; tmp += items[i].weight;
    } else {
        bound += (long long)(items[i].density * (rem - tmp));
        break;
    }
}
if (bound <= res.optimal_value) return; // 上界剪枝
```

3.3 子集和数问题 — 动态规划

算法思想:

定义 $dp[j]$ 为布尔值表示是否存在子集和为 j 。转移: $dp[j] = dp[j] \text{ or } dp[j-a_i]$ 。通过 parent 数组记录转移来源以回溯方案。O(nt)时间。

伪代码:

- 算法 SubsetSum_DP(numbers[1..n], target)
1. if target = 0 then return (true, empty)
 2. if sum(numbers) < target then return (false, empty)
 3. $dp[0..target] = \text{false}$, $dp[0] = \text{true}$
 4. $parent[0..target] = -1$
 5. for $i = 1$ to n do
 6. $x = \text{numbers}[i]$
 7. for $j = \text{target}$ downto x do
 8. if not $dp[j]$ and $dp[j - x]$ then
 9. $dp[j] = \text{true}$, $parent[j] = i$
 10. if $dp[\text{target}]$ then 从 parent 链回溯方案
 11. return ($dp[\text{target}]$, selected)

关键代码(C++):

```
vector<bool> dp(target + 1, false);
vector<int> parent(target + 1, -1);
```

```

dp[0] = true;
for (int i = 0; i < n; i++) {
    long long x = numbers[i];
    if (x > target) continue;
    for (long long j = target; j >= x; j--)
        if (!dp[j] && dp[j - x])
            { dp[j] = true; parent[j] = i; }
}
// 回溯方案:沿 parent 链逆推
if (dp[target]) {
    long long cur = target;
    while (cur > 0 && parent[cur] != -1) {
        selected.push_back(parent[cur]);
        cur -= numbers[parent[cur]];
    }
}

```

3.4 子集和数问题 — 回溯法

算法思想:

数字从大到小排序(大数优先加速剪枝)。剪枝: (1)cur_sum>target 剪枝;
(2)cur_sum+suffix[idx]<target 剪枝(剩余全选也不够); (3)找到解立即终止。

伪代码:

算法 SubsetSum_Backtrack(numbers[1..n], target)

1. numbers = 降序排列, 预计算后缀和 suffix[0..n]
2. found = false
3. procedure DFS(idx, cur_sum)
4. if cur_sum = target then found = true; return
5. if cur_sum > target or idx > n then return
6. if cur_sum + suffix[idx] < target then return //后缀和剪枝
7. DFS(idx+1, cur_sum + a[idx]) //选
8. if found then return
9. DFS(idx+1, cur_sum) //不选
10. DFS(1, 0)
11. return found

关键代码(C++):

```

void dfs(vector<Elem>& elems, vector<long long>& suffix,
        long long target, int idx, long long cur_sum, ...) {

```

```

    if (cur_sum == target) { found = true; return; }
    if (cur_sum > target || idx >= n) return;
    if (cur_sum + suffix[idx] < target) return; //后缀和剪枝
    if (cur_sum + elems[idx].val <= target)    //左分支:选
        dfs(elems, suffix, target, idx+1,
            cur_sum + elems[idx].val, ...);
    if (found) return;
    dfs(elems, suffix, target, idx+1, cur_sum, ...); //右分支
}

```

3.5 矩阵乘法链问题 — 动态规划

算法思想:

$dp[i][j] = \min_{i \leq k < j} \{dp[i][k] + dp[k+1][j] + d[i-1]*d[k]*d[j]\}$ 。通过 $split[i][j]$ 记录最优分割点重建括号化方案。 $O(n^3)$ 时间。

伪代码:

```

算法 MatrixChain_DP(dims[0..n])
1. for i = 1 to n do dp[i][i] = 0
2. for len = 2 to n do
3.   for i = 1 to n-len+1 do
4.     j = i + len - 1, dp[i][j] = INF
5.     for k = i to j-1 do
6.       cost = dp[i][k] + dp[k+1][j]
7.         + dims[i-1]*dims[k]*dims[j]
8.       if cost < dp[i][j] then
9.         dp[i][j] = cost, split[i][j] = k
10. return dp[1][n], BuildParen(split, 1, n)

```

关键代码(C++):

```

for (int len = 2; len <= n; len++) {
    for (int i = 1; i <= n - len + 1; i++) {
        int j = i + len - 1;
        dp[i][j] = LLONG_MAX;
        for (int k = i; k < j; k++) {
            long long cost = dp[i][k] + dp[k+1][j]
                + (long long)dims[i-1]*dims[k]*dims[j];
            if (cost < dp[i][j])
                { dp[i][j] = cost; split[i][j] = k; }
        }
    }
}

```



```

    }
}
// 递归构造括号化: BuildParen(i,j)=
// "(" + BuildParen(i,k) + BuildParen(k+1,j) + ")"

```

3.6 矩阵乘法链问题 — 回溯法

算法思想:

回溯搜索所有加括号方式(Catalan($n-1$)种)。先用贪心启发式(每次选最小中间维度拆分)获得初始上界, 再通过下界矩阵 $lb[i][j]$ 剪枝。子问题间传值约束上界。 $n > 18$ 时跳过。

伪代码:

```

算法 MatrixChain_Backtrack(dims[0..n])
1. 预计算  $lb[i][j]$  = 乐观下界估计
2. upper = GreedyUB(dims, 1, n)    //贪心初始上界
3. procedure Search(i, j, bound)
4.   if  $i = j$  then return 0
5.   best = bound
6.   for  $k = i$  to  $j-1$  (从中间向外) do
7.     if  $lb[i][k] + lb[k+1][j] + \text{merge} \geq \text{best}$  continue
8.     left = Search(i, k, best -  $lb[k+1][j]$  - merge)
9.     if left  $\geq$  best continue
10.    right = Search(k+1, j, best - left - merge)
11.    best = min(best, left + right + merge)
12.   return best
13. return Search(1, n, upper)

```

关键代码(C++):

```

// 贪心启发式:每次选最小中间维度拆分
long long greedy_upper_bound(dims, i, j) {
    if (i == j) return 0;
    int best_k = i;
    for (int k = i; k < j; k++)
        if (dims[k] < dims[best_k]) best_k = k;
    return greedy_upper_bound(i, best_k)
        + greedy_upper_bound(best_k+1, j)
        + dims[i-1] * dims[best_k] * dims[j];
}

```

四、实验结果及复杂性分析

4.1 0/1 背包问题

4.1.1 完整实验结果（已通过标准答案验证）

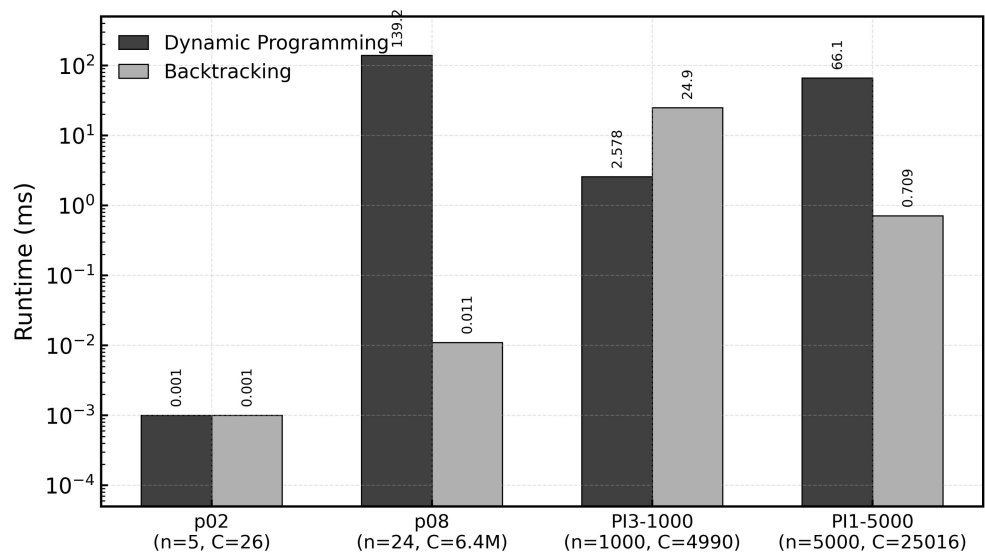


图 1 0/1 背包问题四组代表性实例的 DP 与回溯运行时间对比（对数纵轴）

数据集	n	容量	DP(ms)	回溯 (ms)	DP 操作	回溯节点	最优值	标准答案
p02	5	26	0.001	0.001	14	16	51	51
p08	24	6,404,180	139.157	0.011	144	27	13,549,094	13,549,094
knapPI_3_1000	1000	4,990	2.578	24.904	4,989,000	776	14,390	14,390
knapPI_1_5000	5000	25,016	66.061	0.709	125,064,000	155	276,457	276,457

代表性数据集的最优值与数据集标准答案或两种算法交叉校验结果一致；下列图表主要用于分析不同参数和数据分布下的性能差异。

4.1.2 结果分析

(1) 小规模基准(p02, n=5, C=26)：两种算法均在约 0.001ms 内完成，运行时间低于计时分辨率的稳定区间，因此不宜据此判断优劣；该组主要用于验证两种算法输出与标准答案一致。

(2) 超大容量场景(p08, $n=24$, $C=6,404,180$): DP 耗时 139.2ms, 而回溯约 0.011ms, 回溯约快 12,650 倍。根本原因是 DP 表规模直接受容量 C 控制, 约为 nC 级; 回溯经过价值密度排序和分数背包上界剪枝后, 仅需探索很少节点即可确认最优解。该组说明: 当容量远大于物品总量或剪枝上界很紧时, 回溯能绕开伪多项式 DP 的容量瓶颈。

(3) 强相关实例(knapPI_3_1000, $n=1000$, $C=4,990$): DP 耗时 2.6ms, 回溯耗时 24.9ms, DP 约快 9.7 倍。强相关数据中物品价值与重量关系接近, 分数背包上界对最优整数解的区分度下降, 导致回溯剪枝收益减弱。从 knapPI 序列看, 随着 n 增大, 两种算法均增长, 但回溯曲线受数据分布影响更剧烈; 因此不能只用 n 判断算法快慢, 还必须结合相关性与上界紧致度。

(4) 大规模非相关(knapPI_1_5000, $n=5000$, $C=25,016$): DP 耗时 66.1ms, 回溯约 0.71ms, 回溯约快 93 倍。与强相关实例相比, 非相关数据更容易通过排序和上界剪枝快速排除无效分支, 形成明显的性能反转。该现象表明: 回溯法的实际效率高度依赖数据分布; DP 虽然通常更稳定, 但在容量较大、剪枝很强的实例上并不总是最快。

4.1.3 复杂度验证

DP 操作数与 $O(nC)$ 模型一致, 例如 knapPI_3_1000 约执行 4,989,000 次状态更新, 接近 1000×4990 。回溯节点数则不与 n 或 C 单独线性对应, 而由排序后上界的紧致程度决定: 非相关数据可能很快收敛, 强相关数据则容易出现更多搜索。

4.2 子集和数问题

4.2.1 完整实验结果

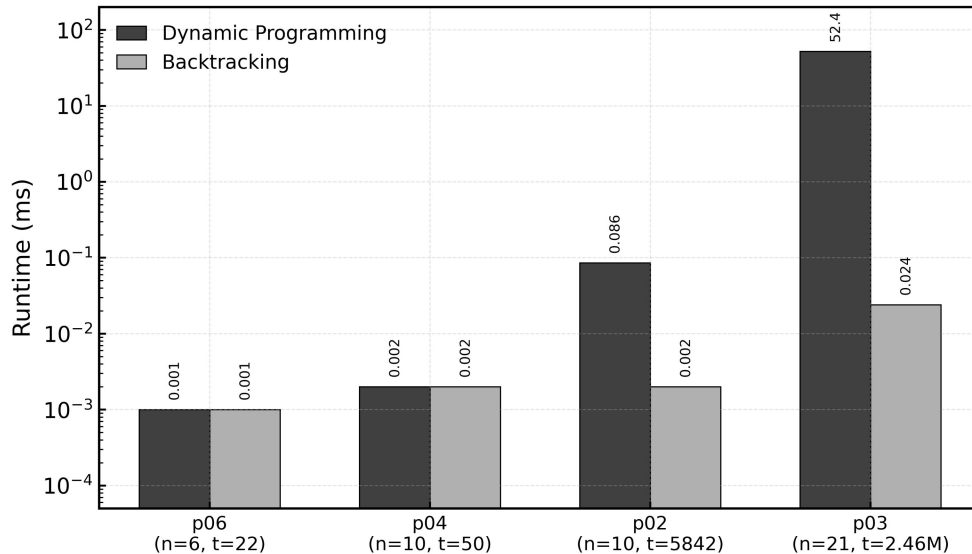


图 2 子集和数问题四组代表性实例的 DP 与回溯运行时间对比 (对数纵轴)

数据集	n	目标和	总和	DP(ms)	回溯 (ms)	DP 操作	回溯节点	子集和验证
p06	6	22	63	0.001	0.001	92	6	OK(sum=22)
p04	10	50	148	0.002	0.002	465	10	OK(sum=50)
p02	10	5,842	11,275	0.086	0.002	58,419	10	OK(sum=5842)
p03	21	2,463,098	19,499,483	52.367	0.024	51,724,947	21	OK(sum=2463098)

4.2.2 结果分析

(1) 目标和小(p06、p04 等): 由于 n 和 t 都较小, DP 表规模有限, 回溯搜索树也很浅, 两者都在微秒级完成。此时算法选择对总时间影响不大, 更应优先考虑实现复杂度和结果可解释性。

(2) 目标和中(p02, $t=5,842$, $n=10$): DP 耗时 0.086ms, 回溯约 0.002ms。DP 需要维护到目标和 t 的状态表, 而回溯在降序搜索与后缀和剪枝帮助下较快找到可行解, 因此在该实例上更快。

(3) 极端大目标(p03, $t=2,463,098$, $n=21$): DP 耗时 52.4ms, 回溯约 0.024ms。DP 时间随 t 显著上升, 符合 $O(nt)$ 伪多项式复杂度的预期; 回溯没有建立按 t 展开的状态表, 在该数据上受目标值大小影响较小。需要注意, 这并不表示回溯对所有大 t 实例都稳定, 只说明本组数据的剪枝条件非常有效。

结论: 目标和 t 是子集和 DP 的关键瓶颈参数; 当 t 很大而 n 较小、且排序或后缀和剪枝能够迅速定位可行解时, 回溯可能显著快于 DP。但回溯仍保留指数级最坏情况, 不能简单概括为“对 t 完全不敏感”。

4.3 矩阵乘法链问题

4.3.1 完整实验结果

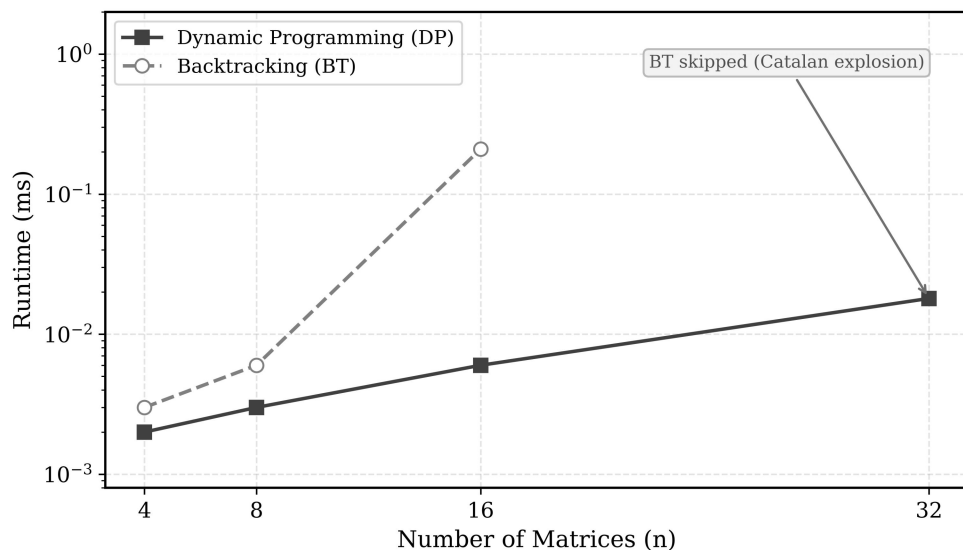


图 3 矩阵乘法链问题中 DP 与回溯随 n 增长的运行时间变化（对数纵轴，n=32 处回溯因 Catalan 数爆炸而跳过）

数据集	n	维度范围	DP(ms)	回溯(ms)	DP 操作	回溯节点	最优值
input_004	4	[2,14]	0.002	0.003	10	3	572
input_008	8	[1,16]	0.003	0.006	84	33	496
input_016	16	[2,13]	0.006	0.210	680	5,631	2,048
input_032	32	[1,16]	0.018	-	5,456	-	1,459

注: $n \geq 32$ 时回溯因 Catalan 数爆炸($C_{31} \sim 1.45 \times 10^{16}$)直接跳过，标记为"-"。

4.3.2 结果分析

(1) $n=4$: DP 和回溯都能瞬时完成，差异主要来自常数开销；该组用于验证括号化代价计算正确。

(2) $n=8$: 回溯从理论 429 种括号化中通过剪枝仅访问少量节点，仍能快速完成；DP 按区间长度填表，操作数为 84，增长规律清晰。

(3) $n=16$: 回溯访问 5,631 个节点、耗时 0.210ms，明显高于 DP 的 0.006ms。虽然剪枝把 $Catalan(15)=9,694,845$ 的理论空间大幅压缩，但剩余搜索量已经超过 DP 的三重循环成本。

(4) $n=32$: DP 仅需 5,456 次状态转移并在 0.018ms 左右完成；回溯因 $Catalan(31)$ 约 1.45×10^{16} ，实验中直接跳过。矩阵链问题的最优子结构和重叠子问题非常典型，DP 在规模增长时具有决定性优势。

4.3.3 复杂度验证

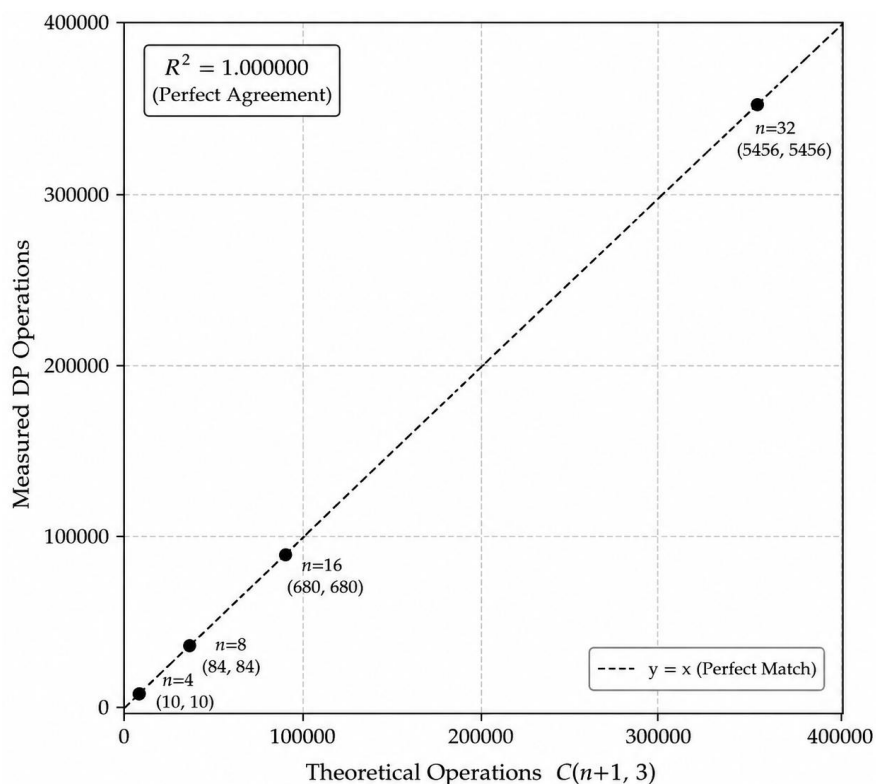


图 4 矩阵乘法链 DP 实测操作数与理论值 $C(n+1,3)$ 的重合验证

DP 操作数与组合数公式 $C(n+1,3)=n(n-1)(n+1)/6$ 完全一致： $n=4、8、16、32、64、128$ 的实测值均落在理论曲线上。这说明程序的区间 DP 循环结构正确，也从实验层面支撑了 $O(n^3)$ 复杂度结论。

4.4 综合分析

4.4.1 数据集特征对算法性能的影响

影响因素	动态规划	回溯法	优势方
问题规模 n	多项式增长	指数级(最坏)	DP(n 大时)
容量/目标和 C/t	线性依赖 $O(nC)/O(nt)$	不直接依赖	回溯(C/t 大时)
数据相关性	不敏感	高度敏感(影响剪枝)	相关:DP; 非相关:回溯
约束松紧	不敏感	高度敏感	松约束:回溯
搜索空间结构	无影响	决定剪枝上限	链状问题:DP(Catalan 爆炸)

4.4.2 算法选择建议

动态规划适用：容量或目标和处于可接受范围、需要稳定可预测的运行时间、或问题具有明确的重叠子问题结构时优先选择。尤其是矩阵链这类区间 DP 问题，DP 把指数级括号化空间压缩为多项式状态转移，优势最明显。

回溯法适用： n 不大或数据分布使剪枝上界很紧、只需寻找可行解或可快速获得高质量初始解时优先考虑。其优势是可能避开 C/t 带来的 DP 表规模膨胀；风险是剪枝失效后会退化到指数级搜索。

五、总结

本实验针对 0/1 背包、子集和数和矩阵乘法链三个经典问题，实现并比较了动态规划与回溯法。代表性实验结果与数据集标准答案或两种算法相互校验结果一致，说明实现具有较好的正确性；同时，通过不同规模与分布的数据集展示了伪多项式 DP、剪枝搜索和区间 DP 三类性能机制的差异。

主要结论：(1) 动态规划的耗时主要由状态规模决定，背包和子集和分别受容量 C 、目标和 t 影响，矩阵链则呈稳定 $O(n^3)$ 增长；(2) 回溯法的优势来自剪枝质量，在非相关或约束较松的数据上可能比 DP 快几个数量级，但在强相关或 Catalan 级搜索空间中会迅速失去优势；(3) 算法选择不能只看问题名称，应同时判断 n 、 C/t 、数据相关性、上界紧致度和是否存在大量重叠子问题。

通过本实验可以看出，实验结论应建立在“参数规模 + 数据分布 + 理论复杂度”共同解释之上。DP 的价值在于稳定复用子问题，回溯的价值在于利用实例结构减少搜索；二者不是简单的快慢关系，而是在不同数据特征下各有适用边界。